

Strategy Validator: A Risk-Based Model for Validating Trading Strategies using Monte Carlo Simulation and the RSB Approach

Kriss

<https://github.com/k-means-kriss/Strategy-Drawdown-Validator-using-RSB-Monte-Carlo-Simulation>

Abstract

Trading strategies inevitably go through drawdowns, and a central practical question is whether an observed drawdown is a normal statistical fluctuation or a genuine sign that the strategy has broken. This paper presents a risk-based validation framework that combines Monte Carlo simulation with the RSB (Rej–Seager–Bouchaud) drawdown model to benchmark an observed “broken” segment of a strategy’s return series against thousands of simulated healthy paths. Three simulation engines — Gaussian, empirical (bootstrap), and RSB — are used to generate reference distributions for five risk statistics: maximum drawdown, maximum loss, days spent underwater, recovery time, and the most recent negative streak. The observed statistics are converted to percentile scores against these distributions and mapped to a four-tier health classification (Healthy, Normal, Borderline, Breaking). The framework is demonstrated on both a synthetic one-year dataset and a ten-year volatility-targeted momentum strategy on RELIANCE.NS.

1 Introduction

Every systematic strategy eventually loses money for a while. The difficulty is deciding, in real time, whether a given losing streak is ordinary noise around the strategy’s assumed Sharpe ratio or evidence that the strategy’s edge has genuinely deteriorated. Relying on a single number such as maximum drawdown is fragile: two strategies can share the same peak-to-trough loss while differing enormously in how long they stayed underwater or how recently the pain occurred. This project builds a small, reproducible validator that (i) simulates many alternate “healthy” futures for a strategy under different statistical assumptions, (ii) extracts five complementary risk statistics from both the simulations and the real (possibly broken) segment, and (iii) reports where the real segment falls in the simulated distribution, turning that percentile into a plain Healthy/Normal/Borderline/Breaking verdict.

2 Literature Review

The framework is inspired by two papers. *You are in a drawdown. When should you start worrying?* (Rej, Seager & Bouchaud, Capital Fund Management) models a strategy’s cumulative P&L as a drifted Brownian motion and derives analytical thresholds for how deep and how long a drawdown can be before it becomes statistically alarming for a strategy of a given Sharpe ratio. *Drawdown Risk Beyond Brownian Motion: A Monte-Carlo Framework, Non-Gaussian Extensions, and Long Memory* (Landolfi) extends this idea numerically, showing that Monte Carlo resampling of returns — rather than relying purely on closed-form Brownian results — gives a more flexible and realistic picture of drawdown risk once fat tails and empirical return structure are taken

into account. This project borrows the drifted-Brownian-motion core from the first paper and the simulation-driven, non-Gaussian validation philosophy from the second, combining both with a simple empirical (bootstrap) resampling engine to triangulate risk.

3 What is the RSB Model?

The RSB (Rej–Seager–Bouchaud) model asks whether a trading strategy’s drawdown is statistically normal or unusually severe given its assumed Sharpe ratio. It models the strategy’s P&L as a drifted Brownian motion,

$$dPnL = \mu dt + \sigma dW,$$

where the drift μ represents the strategy’s Sharpe ratio once volatility has been normalized away. The model focuses on two quantities: drawdown depth (how far the strategy falls from its peak) and drawdown length (how long it stays down), and provides probability distributions and 5% extreme thresholds for both. If the strategy’s realized drawdown depth or length exceeds these thresholds, it is a warning sign that the assumed Sharpe ratio, or the strategy itself, needs to be re-examined. In simple terms: RSB describes how much and how long a strategy can normally suffer for a given Sharpe ratio, and flags drawdowns that fall outside that normal range.

4 Necessary Libraries and Data Import

The following packages are required. `ta` and `mplfinance` are installed via `pip` since they are not part of the standard scientific stack.

```
!pip install ta mplfinance
```

```
import pandas as pd
import yfinance as yf
import numpy as np
import matplotlib.pyplot as plt
import time
import ta
import mplfinance as mpl
```

Data can be supplied either as a CSV file or as a ticker symbol, in which case one year of daily history is pulled from Yahoo Finance and daily returns are computed from the close price:

```
def Data():
    print('you can either give ur data or enter ticker of the asset')
    time.sleep(2)
    data=input()
    if data.endswith('.csv'):
        Data=pd.read_csv(data)
    else:
        ticker=yf.Ticker(data)
        df=ticker.history(period='1y')
        df['returns']=df['Close'].shift(-1)-df['Close']
        Data=(df[['Close','returns']])
    print(Data.head())
    return Data
```

5 Risk Measurement Functions

Five functions each take a list of price paths (simulated or real) and extract one risk statistic per path. All five track the running peak of the path and measure some aspect of behavior relative to that peak. Using all five together, rather than drawdown alone, gives a fuller picture of how a strategy misbehaves rather than just whether it does.

5.1 Maximum Drawdown — `drawdown()`

Tracks the running peak at every point in time and records the largest peak-to-trough gap observed across the path: the worst losing streak in price terms.

```
def drawdown(l):
    s=[]
    subs=[]
    for i in l:
        f=[]
        diff=[]
        maxi=0
        for j in range(len(i)):
            if j==0:
                maxi=i[j]
                dd=maxi-(i[j])
                f.append(dd)
            elif i[j]>=maxi:
                maxi=i[j]
                dd=maxi-(i[j])
                f.append(dd)
            elif i[j]<maxi:
                dd=maxi-(i[j])
                f.append(dd)
        s.append(max(f))
    return s
```

5.2 Maximum Loss — `max_loss()`

Measures the largest cumulative drop in the running sum of returns, regardless of whether the low comes after the high in time; a broader, order-agnostic loss measure than drawdown.

```
def max_loss(gauss):
    maxi=[]
    for i in gauss:
        series=pd.Series(i)
        returns=series.shift(-1)-(series)
        returns = returns.dropna()
        sums=[]
        sum=0
        for i in returns:
            sum+=i
            sums.append(sum)
        maxi.append(max(0, -min(sums)))
    return maxi
```

5.3 Final Negative Time — days_of_lower()

Counts how many days out of the whole path the price spends below its running peak (“underwater”). A high value means the strategy spent most of its life recovering rather than sitting at new highs.

```
def days_of_lower(l):
    count=[]
    for i in l:
        maxi=0
        c=0
        for j in range(len(i)):
            if i[j]>=maxi:
                maxi=i[j]
            elif i[j]<maxi:
                c+=1
        count.append(c)
    return count
```

5.4 Longest Recovery Time — recovery_days()

Finds the longest unbroken stretch of consecutive underwater days, capturing duration rather than depth.

```
def recovery_days(l):
    count = []
    for i in l:
        maxi = 0
        c = 0
        f = []
        for j in range(len(i)):
            if i[j] >= maxi:
                maxi = i[j]
                f.append(c)
                c = 0
            elif i[j] < maxi:
                c += 1
        f.append(c)
        count.append(max(f))
    return count
```

5.5 Last Negative Streak — last_neg()

Tracks a running cumulative sum of daily returns and records the last index at which that sum was negative, capturing how recently (and for how long, near the end) the strategy was losing.

```
def last_neg(gauss):
    neg = []
    for i in gauss:
        series = pd.Series(i)
        returns = series.diff().dropna()
        streak = 0
        last_streak_end = 0
        for j in range(len(returns)):
            if returns.iloc[j] < 0:
                streak += 1
```

```

        last_streak_end = j
    else:
        streak = 0
    neg.append(last_streak_end if streak > 0 or last_streak_end > 0 else 0)
return neg

```

6 Strategy Validation Rules

Each risk statistic is converted into a percentile against its simulated distribution and mapped to a health tier as in Table 1. If any critical risk metric exceeds the 95th percentile, the overall strategy is classified as Breaking.

Percentile Range	Health Status	Interpretation
< 50th	Healthy	Better than most Monte Carlo outcomes
50th–90th	Normal	Within the expected range
90th–95th	Borderline	Deteriorating, elevated risk
> 95th	Breaking	Significantly worse than expected

Table 1: Strategy validation rules.

7 Data Engineering: Healthy/Broken Split

Before any model can be run, the dataset must be manually split into a *healthy* segment (used to estimate the strategy’s statistical parameters) and a *broken* segment (the suspicious tail to be tested). The same procedure is used for both the 1-year and the 10-year model; only the input dataset and the chosen breakdown point differ.

```

import pandas as pd

# Load the 1-year synthetic strategy data
returns = data['returns'].values
close=data['Close'].values

# Known breakdown point (engineered at day 150 for the 1yr dataset)
breakdown_point = int(input('enter the breaking point:'))
time.sleep(2)

# Split into healthy and broken segments
healthy = returns[:breakdown_point]
healthy_close=close[:breakdown_point]
broken = returns[breakdown_point:]
broken_close=close[breakdown_point:]

print("Healthy days:", len(healthy))
print("Broken days:", len(broken))

```

For the demonstration dataset this yields 150 healthy days and 102 broken days (Fig. 1), confirming that the engineered breakpoint was picked up correctly.

```

enter the breaking point : 150
Healthy days: 150
Broken days: 102

```

Figure 1: Console output of the healthy/broken split at the engineered breakdown point.

8 One-Year Model

8.1 Exploratory Plots

Two visual diagnostics are produced before simulation. `returns_plot` draws the raw return series (Fig. 2), and `plot_analyser` draws a 2×2 grid (Fig. 3) containing: (1) the closing price series, (2) the returns series, (3) the healthy and broken segments overlaid on the same time axis, and (4) the same two segments placed end-to-end on a continuous time axis. Together these four panels show the raw asset behavior, the volatility of returns, and exactly where and how the visual character of the series changes once the strategy enters the broken regime – the broken segment (Fig. 3, bottom panels) visibly trends down where the healthy segment is flat/noisy.

```

def returns_plot(data):
    plt.plot(data['returns'])

```

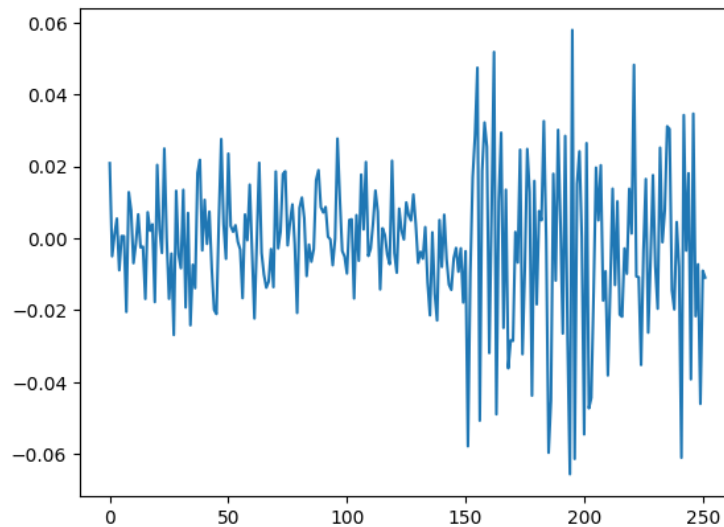


Figure 2: 1-year returns plot.

```

def plot_analyser(data,broken,healthy):
    fig, ax = plt.subplots(2, 2, figsize=(12, 8))
    ax[0, 0].plot(data['Close']); ax[0, 0].set_title('Closing Price of the Asset')
    ax[0, 1].plot(data['returns']); ax[0, 1].set_title('Returns Plot')
    ax[1, 0].plot(healthy, label="healthy"); ax[1, 0].plot(broken, label="broken")
    ax[1, 0].set_title('BROKEN vs HEALTHY'); ax[1, 0].legend()
    ax[1, 1].plot(range(len(healthy)), healthy, label="healthy")
    ax[1, 1].plot(range(len(healthy), len(healthy)+len(broken)), broken, label="broken")
    ax[1, 1].set_title('BROKEN and HEALTHY in a Continuous Time Frame'); ax[1, 1].legend()
    plt.tight_layout(); plt.show()

```

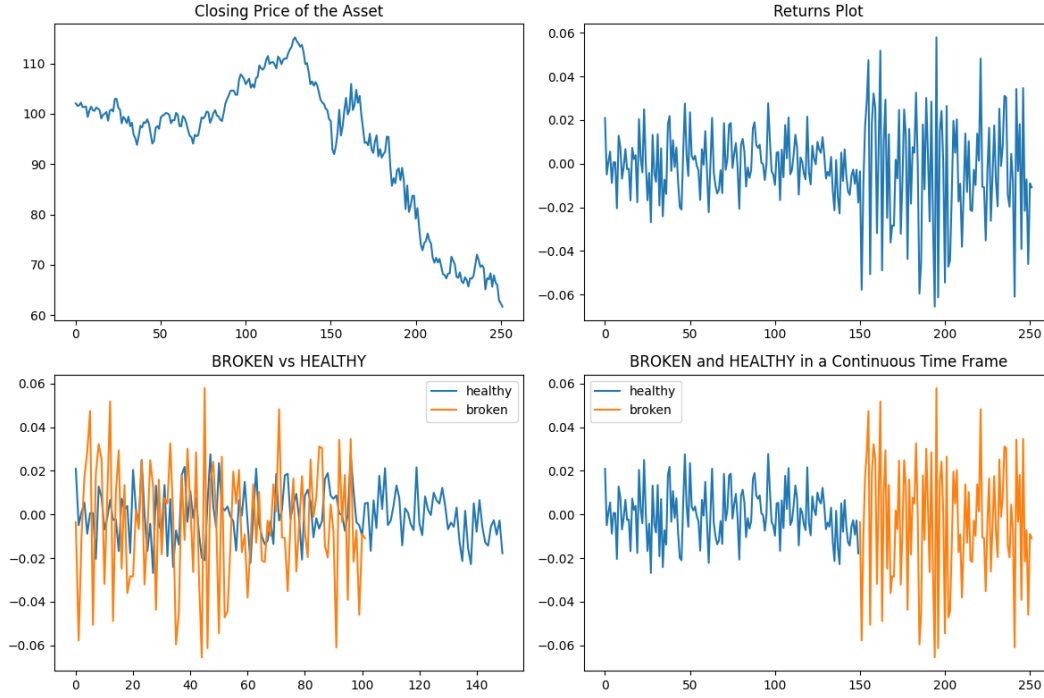


Figure 3: Four-panel diagnostic: price, returns, healthy vs. broken, continuous timeline.

8.2 Calculation Terms

The healthy segment is used to estimate the drift, volatility, and Sharpe ratio that seed the simulations:

```
days = 252
total_simulations = 10000
simulations = [[0] * days for _ in range(total_simulations)]
last_price = data['Close'].iloc[breakdown_point - 1]
avg_returns = healthy.mean()
volatility = healthy.std()
mu = avg_returns * days
standard_volatility = volatility * np.sqrt(days)
sharpe = avg_returns / volatility
annual_sharpe = sharpe * np.sqrt(252)
sharpe, avg_returns, volatility, annual_sharpe
```

For the demonstration data this returns a daily Sharpe of -0.00022 , mean return -2.56×10^{-6} , volatility 0.01163, and annualized Sharpe -0.00349 – a near-flat healthy segment, consistent with the flat left half seen in Fig. 3.

8.3 Monte Carlo – Gaussian

The Gaussian engine simulates 10,000 alternative price paths by drawing i.i.d. normal daily returns with the healthy segment’s mean and volatility.

```
def gauss_1yr(total_simulations, days, avg_returns, volatility, last_price):
    l=[]
    for i in range(total_simulations):
        current_simulated_price = last_price
```

```

f=[]
for j in range(days):
    daily_random_return = np.random.normal(loc=avg_returns, scale=volatility)
    current_simulated_price = current_simulated_price * (1 + daily_random_return)
    f.append(current_simulated_price)
l.append(f)
return l

```

```

def gauss_graph(l):
    for i in l:
        plt.plot(i)
    plt.title('MONTE CARLO - GAUSSIAN')
    plt.show()

```

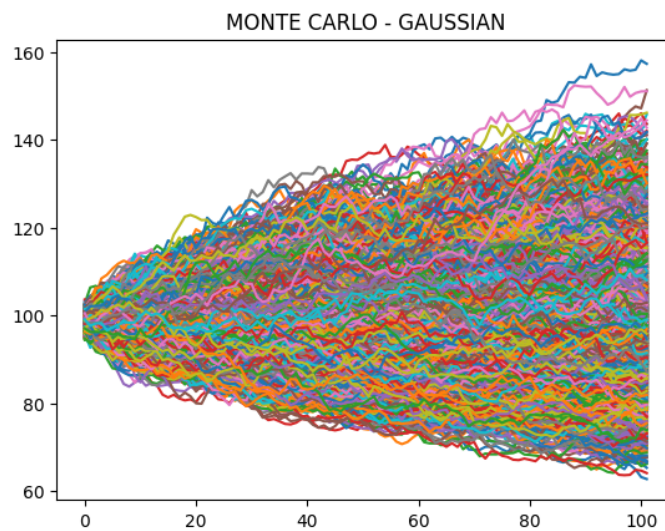


Figure 4: 10,000 simulated Gaussian price paths over the broken-segment horizon.

```

def gauss_distribution(l):
    final_prices = []
    for simulation in l:
        final_prices.append(simulation[-1])
    plt.hist(final_prices, bins=80)
    plt.xlabel("Final Price"); plt.ylabel("Number of Simulations")
    plt.title("Monte Carlo Distribution"); plt.show()

```

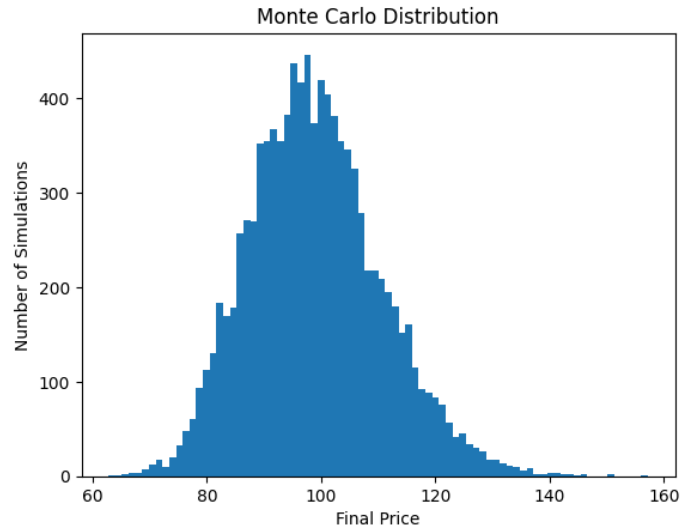



Figure 5: Distribution of final simulated prices, Gaussian engine.

The fan of paths in the first plot spreads out roughly symmetrically since returns are drawn from a normal distribution; the histogram of terminal prices is correspondingly bell-shaped and centered near the last observed price, since the estimated drift is close to zero.

8.4 Monte Carlo – Empirical

The empirical engine instead bootstraps daily returns directly from the historical healthy sample, preserving the true (non-Gaussian) shape of the return distribution.

```
def emperical_1yr(total_simulations,days,data,avg_returns,volatility,last_price):
    c=[]
    returns=data['returns']
    for i in range(total_simulations):
        current_simulated_price = last_price
        f=[]
        for j in range(days):
            daily_random_return = float(np.random.choice(returns))
            current_simulated_price = current_simulated_price * (1 + daily_random_return)
            f.append(current_simulated_price)
        c.append(f)
    return c
```

```
def emperical_graph(c):
    for i in c:
        plt.plot(i)
    plt.title('MONTE_CARLO-EMPERICAL')
    plt.show()
```

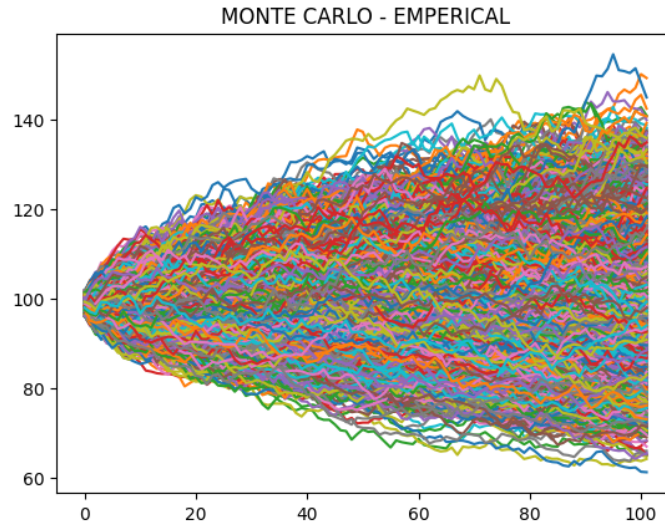


Figure 6: 10,000 simulated empirical (bootstrapped) price paths.

```
def emperical_distribution(c):
    final_prices = []
    for simulation in c:
        final_prices.append(simulation[-1])
    plt.hist(final_prices, bins=80)
    plt.xlabel("Final_Price"); plt.ylabel("Number_of_Simulations")
    plt.title("Monte_Carlo_Distribution"); plt.show()
```

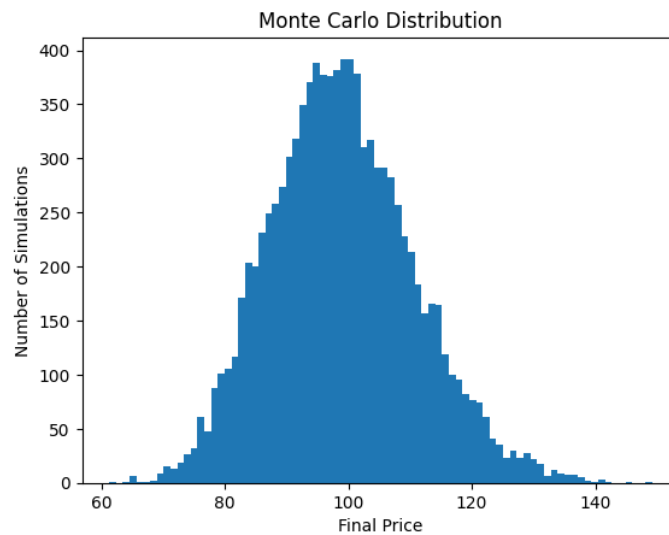


Figure 7: Distribution of final simulated prices, empirical engine.

The empirical fan and histogram closely track the Gaussian ones here because the healthy segment itself is close to normally distributed, but this engine would diverge visibly whenever the real returns are fat-tailed or skewed.

8.5 One-Year Results

Three data frames summarize the one-year model. First, the 50th/90th/95th percentiles of each risk statistic across the Gaussian and empirical simulations (Table 2) – these define the reference scale that the real broken segment is measured against.

	50th	90th	95th
gaussian_drawdown	12.37	20.61	23.12
real_drawdown	12.22	20.53	23.18
gauss_days	93.00	100.00	101.00
real_days	93.00	100.00	101.00
gauss_recovery	64.00	99.00	101.00
real_recovery	64.00	99.00	101.00
gauss_loss	7.27	17.28	20.11
real_loss	7.10	17.36	20.23
gauss_neg	100.00	100.00	100.00
real_neg	100.00	100.00	100.00

Table 2: Simulated risk-statistic percentiles, 1-year model.

Second, the raw risk statistics computed directly on the real broken price path (Table 3) – this is the actual outcome being tested.

Broken_drawdown	Max_Loss	Days_Of_Recovery	Days_Of_Lower	Last_Negative
44.27	36.93	89	98	100

Table 3: Raw risk statistics of the observed broken segment, 1-year model.

Third, those same broken-segment statistics expressed as percentiles against the Gaussian and empirical Monte Carlo distributions (Table 4).

	Gaussian MC	Realistic MC
Drawdown	100.0	100.0
Max Loss	100.0	99.99
Days Lower	79.64	78.90
Recovery	75.58	74.95
Final Negative	74.94	74.18

Table 4: Percentile scores of the observed broken segment, 1-year model.

Model report. Applying the strategy validation rules of Table 1 to Table 4: Drawdown sits at the 100th percentile against both engines, well above the 95th-percentile Breaking threshold, and Max Loss is effectively tied with it. Days Lower, Recovery, and Final Negative all sit in the 74–80th percentile range, i.e. Normal. Because at least one critical metric (drawdown) exceeds the 95th percentile, the overall one-year strategy is classified as **Breaking** – the engineered breakdown at day 150 is correctly detected as a statistically extreme, not merely unlucky, event.

9 Ten-Year Model (RSB Approach)

9.1 Example Strategy

The 10-year test case is a volatility-targeted momentum strategy on RELIANCE.NS: a 126-day momentum signal sets long/short exposure, position size is scaled to target constant 15% annualized volatility (capped at 2x leverage), and transaction costs are charged on every position change.

```
import yfinance as yf
import pandas as pd
import numpy as np

def generate_vol_targeted_momentum_10yr(ticker="RELIANCE.NS", start="2014-01-01", end="2024-01-01",
                                       lookback=126, vol_window=20, target_vol=0.15,
                                       max_leverage=2.0, txn_cost_bps=5):

    data = yf.download(ticker, start=start, end=end)
    close = data['Close'].squeeze()
    daily_returns = close.pct_change()

    momentum = close.pct_change(lookback)
    raw_signal = np.sign(momentum)

    realized_vol = daily_returns.rolling(vol_window).std() * np.sqrt(252)
    position_size = (target_vol / realized_vol).clip(upper=max_leverage)
    position = (raw_signal * position_size).shift(1).fillna(0)

    position_change = position.diff().abs().fillna(0)
    txn_costs = position_change * (txn_cost_bps / 10000)

    strategy_returns = (position * daily_returns) - txn_costs
    strategy_returns = strategy_returns.fillna(0)

    df = pd.DataFrame({'close': close, 'returns': strategy_returns}).dropna()
    return df

df_momentum_10yr = generate_vol_targeted_momentum_10yr()
df_momentum_10yr.to_csv('vol_targeted_momentum_10yr.csv')
df_momentum_10yr
```

The same `Data()`, `healthy/broken split`, `returns_plot`, and `plot_analyser` routines from Sections 4 and 7 are reused unchanged on this 10-year series (only the column name `close` is renamed to `Close`), giving Fig. 8 and Fig. 9.

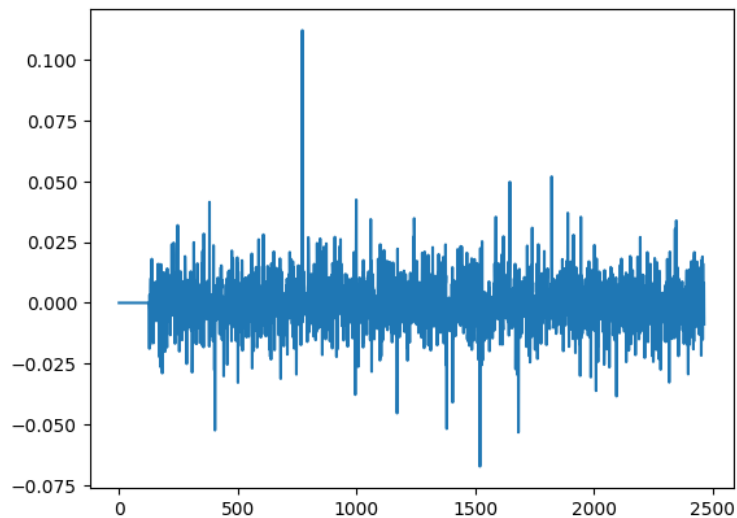


Figure 8: 10-year strategy returns plot.

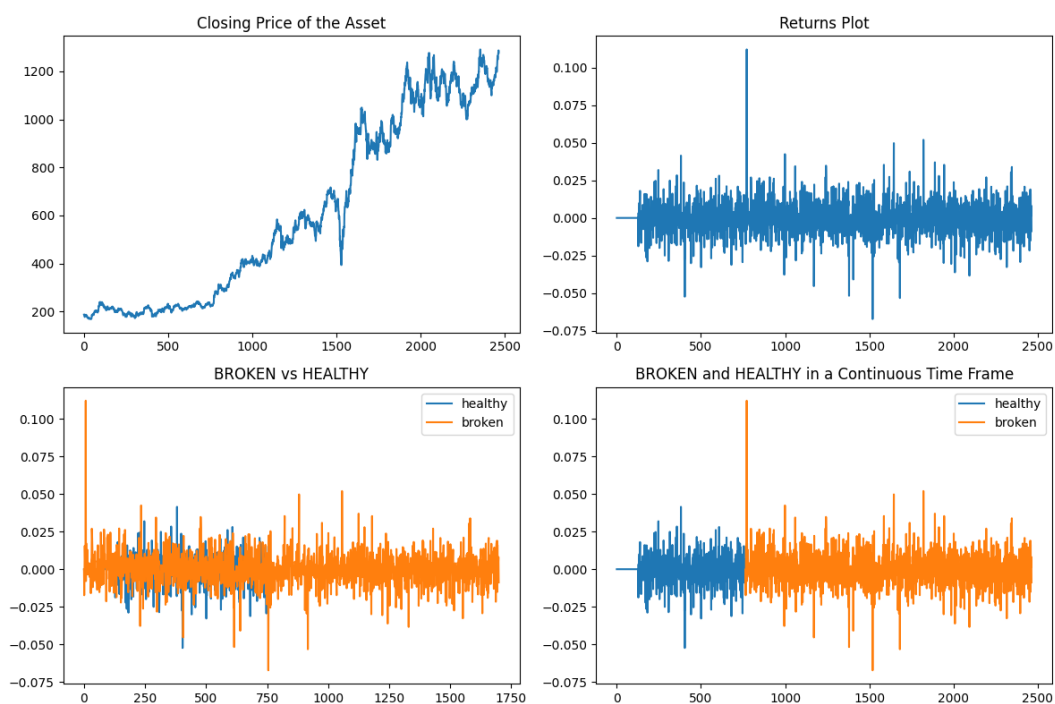


Figure 9: Four-panel diagnostic for the 10-year momentum strategy: price, returns, healthy vs. broken, continuous timeline.

Over ten years the volatility clusters and drawdown episodes are far more visible than in the synthetic 1-year series, and the broken tail (bottom panels) shows a sustained decline rather than a single sharp shock.

9.2 Calculation Terms

With a 10-year (2520 trading day) horizon, the same drift/volatility/Sharpe estimation is repeated on the healthy segment:

```
days = 2520
total_simulations = 10000
simulations = [[0] * days for _ in range(total_simulations)]
last_price = data['Close'].iloc[breakdown_point - 1]
avg_returns = healthy.mean()
volatility = healthy.std()
mu = avg_returns * days
standard_volatility = volatility * np.sqrt(days)
sharpe = avg_returns / volatility
annual_sharpe = sharpe * np.sqrt(252)
sharpe, avg_returns, volatility, annual_sharpe
```

This yields a daily Sharpe of -0.0484 , mean return -4.45×10^{-4} , volatility 0.00919 , and annualized Sharpe -0.7686 – a markedly weaker regime than the 1-year case, reflecting the sustained decline in the broken segment.

9.3 RSB Metrics

The RSB paper's closed-form thresholds for typical drawdown depth and length as functions of annualized Sharpe are computed directly:

```
rsb_depth=1.50/annual_sharpe
rsb_length=2.14 / (annual_sharpe ** 2)
rsb_depth,rsb_length
```

With annualized Sharpe -0.7686 this gives depth ≈ -1.95 and length ≈ 3.62 in RSB-normalized units, i.e. a negative Sharpe regime where the closed-form scale itself signals stress before any simulation is run.

9.4 Monte Carlo Simulations

Three engines are run in normalized (RSB) units for the 10-year model: RSB (drifted Brownian motion parametrized directly by Sharpe), normalized Gaussian, and normalized empirical (vectorized bootstrap).

```
def rsb_mc(annual_sharpe, days,total_simulations):
    rsb_daily_drift = annual_sharpe / 252
    rsb_daily_volatility = 1 / np.sqrt(252)
    daily_returns = np.random.normal(loc=rsb_daily_drift, scale=rsb_daily_volatility,
                                     size=(total_simulations, days))
    pnl_paths = np.cumsum(daily_returns, axis=1)
    rsb = pnl_paths.tolist()
    return rsb
```

```
def norm_gauss_mc(avg_returns, volatility, days,total_simulations):
    daily_returns = np.random.normal(loc=avg_returns, scale=volatility,
                                     size=(total_simulations, days))
    normalized_returns = daily_returns / (volatility * np.sqrt(252))
    pnl_paths = np.cumsum(normalized_returns, axis=1)
    gauss_norm = pnl_paths.tolist()
    return gauss_norm
```

```
def emperical_norm_mc(volatility, data, days,total_simulations):
    returns = data['returns'].values
    sampled_returns = np.random.choice(returns, size=(total_simulations, days), replace=True)
    normalized_returns = sampled_returns / (volatility * np.sqrt(252))
    pnl_paths = np.cumsum(normalized_returns, axis=1)
    real_norm = pnl_paths.tolist()
    return real_norm
```

```
gauss_norm = norm_gauss_mc(avg_returns, volatility, len(broken),10000)
real_norm = emperical_norm_mc(volatility, healthy, len(broken),10000)
rsb_sim = rsb_mc(annual_sharpe, len(broken),10000)
```

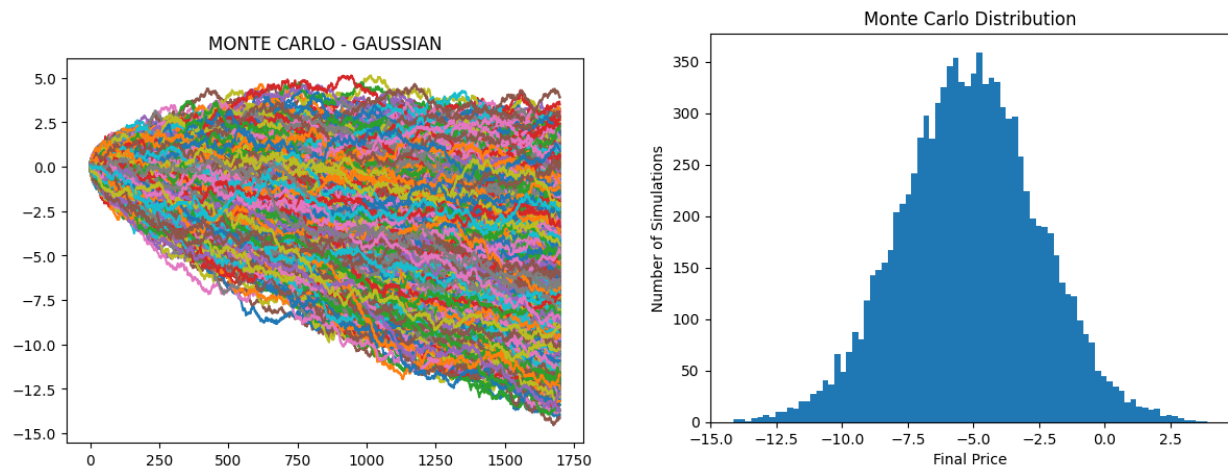


Figure 10: Normalized Gaussian MC paths (left) and terminal-value distribution (right), 10-year model.

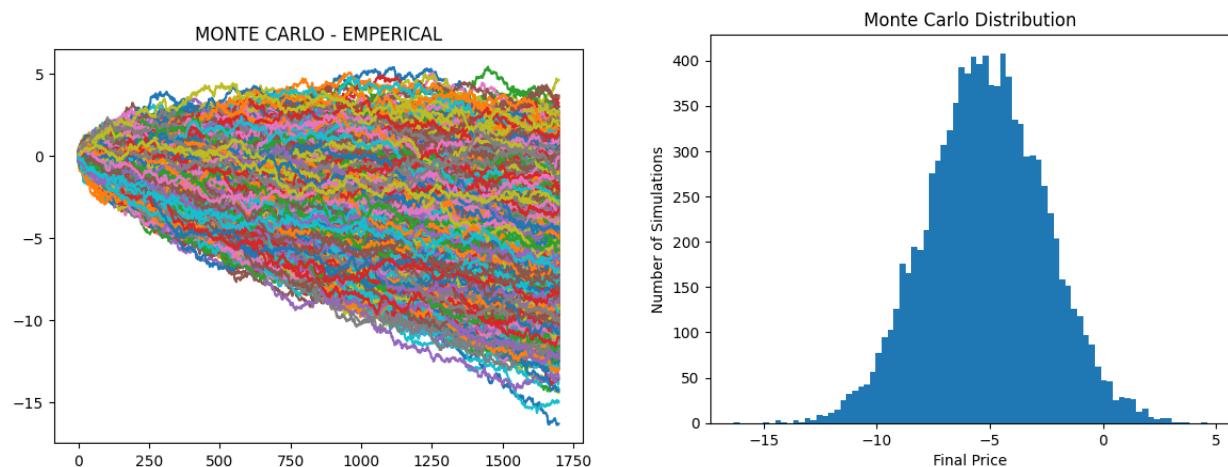


Figure 11: Normalized empirical MC paths (left) and terminal-value distribution (right), 10-year model.

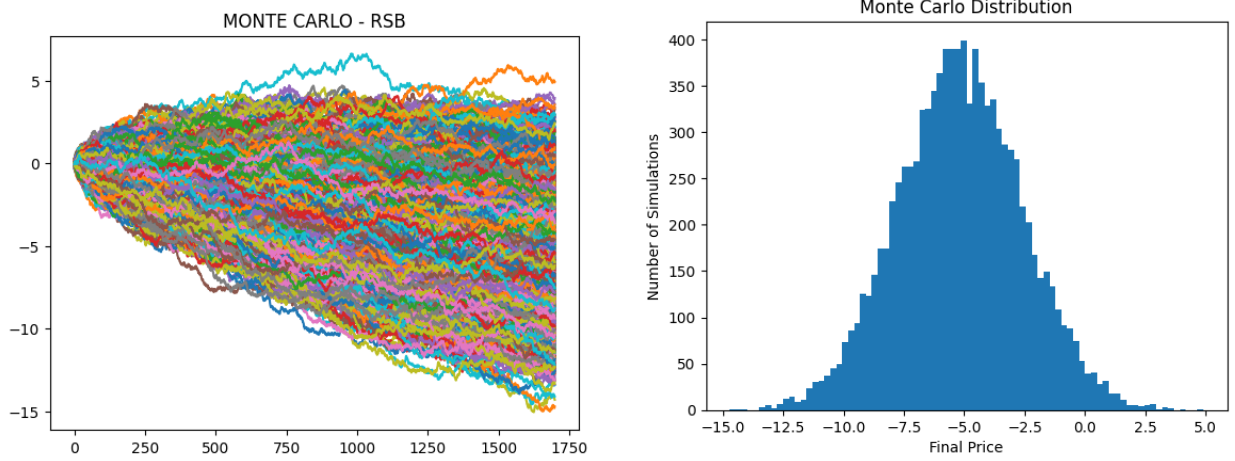


Figure 12: RSB Monte Carlo paths (left) and terminal-value distribution (right), 10-year model.

All three engines produce a strongly negative-drifting fan of paths, consistent with the negative annualized Sharpe estimated above; the RSB engine’s paths are the most tightly clustered since its volatility is fixed by construction ($1/\sqrt{252}$ per day) rather than estimated from data.

9.5 Ten-Year Results: Broken 5

Four data frames summarize the 10-year model. First, the 50th/90th/95th percentiles of each risk statistic across all three engines (Table 5).

	50th	90th	95th
rsb_drawdown	6.22	9.24	10.08
gauss_drawdown	6.22	9.26	10.12
real_drawdown	6.22	9.24	10.13
rsb_days	1690	1699	1700
gauss_days	1690	1699	1700
real_days	1689	1699	1700
rsb_recovery	1633.5	1699	1700
gauss_recovery	1631	1699	1700
real_recovery	1629	1698	1700
rsb_loss	5.73	8.87	9.76
gauss_loss	5.69	8.86	9.79
real_loss	5.71	8.88	9.79
rsb_neg	1698	1698	1698
gauss_neg	1698	1698	1698
real_neg	1697	1698	1698

Table 5: Simulated risk-statistic percentiles, 10-year model.

Second, the raw risk statistics on the real broken price path (Table 6).

Broken_drawdown	Max_Loss	Days_Of_Recovery	Days_Of_Lower	Last_Negative
213.77	27.89	1154	1626	1698

Table 6: Raw risk statistics of the observed broken segment, 10-year model.

Third, the same statistics normalized into RSB units by dividing by $\text{last_price} \times \text{volatility} \times \sqrt{252}$

(Table 7).

Broken_drawdown	Max_Loss	Days_Of_Recovery	Days_Of_Lower	Last_Negative
6.52	0.85	1154	1626	1698

Table 7: Normalized broken-segment risk statistics, 10-year model.

Fourth, those normalized statistics expressed as percentiles against RSB, Gaussian, and empirical Monte Carlo (Table 8).

	RSB MC	Gaussian MC	Realistic MC
Drawdown	55.15	55.33	55.27
Max Loss	1.06	1.18	1.16
Days Lower	0.605	0.465	0.92
Recovery	9.76	10.43	10.38
Final Negative	73.87	74.27	78.61

Table 8: Percentile scores of the observed broken segment, 10-year model.

Model report. Applying the validation rules of Table 1 to Table 8: Drawdown sits around the 55th percentile (Normal). Max Loss and Days Lower are extremely low (under the 1st percentile), meaning the observed segment’s per-day loss size and underwater-day count are far milder than almost every simulated path – also Healthy by the rule (below the 50th percentile). Recovery is likewise low, near the 10th percentile (Healthy). Final Negative sits at 74–79th percentile (Normal). No metric crosses the 90th percentile, so no metric is Borderline or Breaking. The overall 10-year strategy is therefore classified as **Healthy/Normal**: despite a large absolute drawdown in price terms (Table 6), once normalized by the strategy’s own realized volatility and benchmarked against 10,000 simulated alternative histories under the same negative-Sharpe regime, the observed decline is not statistically unusual – the strategy is behaving as its own (poor) Sharpe ratio would predict, rather than showing an additional, abnormal break.

10 Future Work

The healthy/broken split is currently a manual step requiring a user-supplied breakdown point; a natural extension is to automate this data engineering step with an online change-point or regime-detection algorithm so the validator can flag a break as it happens rather than after a point is chosen by hand. Beyond automation, future versions could: extend the RSB closed-form thresholds to non-Brownian, long-memory processes as suggested by Landolfi; add fat-tailed (Student- t or block-bootstrap) simulation engines to better capture volatility clustering; support multi-asset or portfolio-level validation rather than a single return series; and wrap the whole pipeline into a live dashboard that periodically re-scores a running strategy and alerts when any metric crosses the Borderline or Breaking threshold.

References

1. Rej, A., Seager, P., & Bouchaud, J.-P. *You are in a drawdown. When should you start worrying?* Capital Fund Management, 23 rue de l'Université, 75007 Paris.
2. Landolfi, F. *Drawdown Risk Beyond Brownian Motion: A Monte-Carlo Framework, Non-Gaussian Extensions, and Long Memory.* Epiphany, Imperia, Italy.
3. Project code and notebook: <https://github.com/k-means-kriss/Strategy-Drawdown-Validator-using>